

PASSWORDLESS AUTHENTICATION

EC2 Setup via PEM File and Password,
and Passwordless Authentication through Ansible

Professional Reference Documentation

Version 1.0

Table of Contents

1. Introduction to Passwordless Authentication
 - 1.1 What Is Passwordless Authentication?
 - 1.2 How SSH Key-Based Authentication Works
 - 1.3 Why It Matters for Automation
2. EC2 Instance Setup via PEM File
 - 2.1 Creating a Key Pair in AWS
 - 2.2 Setting File Permissions
 - 2.3 Connecting with the PEM File
 - 2.4 Copying the Public Key with ssh-copy-id
3. EC2 Instance Setup via Password
 - 3.1 Enabling Password Authentication
 - 3.2 Setting a Password on the Instance
 - 3.3 Copying the Public Key with a Password
 - 3.4 Testing Passwordless Access
4. Passwordless Authentication through Ansible
 - 4.1 Ansible Inventory Configuration
 - 4.2 Using ansible_ssh_private_key_file
 - 4.3 Using ssh-agent
 - 4.4 Configuring ansible.cfg for Passwordless Access
 - 4.5 Disabling Password Authentication on All Nodes
5. Security Best Practices
6. Troubleshooting
7. Quick Reference

1. Introduction to Passwordless Authentication

1.1 What Is Passwordless Authentication?

Passwordless authentication replaces the traditional username-and-password login with a cryptographic proof of identity. Instead of typing a shared secret that can be guessed, intercepted, or brute-forced, the client proves possession of a private key that never leaves its machine. The server holds only the matching public key, which is useless to an attacker.

In the SSH context, passwordless authentication means the client can connect to a remote server without entering a password. The authentication exchange is automatic and transparent once the key pair is in place.

1.2 How SSH Key-Based Authentication Works

The mechanism follows the SSH Authentication Protocol (RFC 4252):

- **Key generation** — the client generates a mathematically linked key pair: a private key and a public key.
- **Public key distribution** — the public key is appended to `~/.ssh/authorized_keys` on every server the client wants to access.
- **Authentication** — when the client connects, the server issues a challenge. The client signs it with the private key. The server verifies the signature using the public key.
- **Access granted** — if the signature is valid, the server grants access. No password is ever transmitted.

The private key remains on the client machine at all times. The public key is safe to distribute freely.

1.3 Why It Matters for Automation

- **Security** — eliminates brute-force attacks, since there is no password to guess.
- **Automation** — tools like Ansible, Terraform, and CI/CD pipelines cannot type passwords interactively. Key-based authentication is the standard mechanism for unattended access.
- **Scalability** — a single key pair can authenticate to hundreds of servers, and centralised key management is possible.
- **Auditability** — key pairs can be traced to specific users or automation accounts.

2. EC2 Instance Setup via PEM File

When you launch an EC2 instance, AWS can generate a key pair for you. The private key is downloaded as a `.pem` file and is the only copy you will receive. The public key is automatically installed on the instance in the default user's `~/.ssh/authorized_keys`.

2.1 Creating a Key Pair in AWS

- **Via the console** — during instance launch, choose "Create a new key pair", give it a name, and download the `.pem` file.
- **Via the CLI** — use `aws ec2 create-key-pair`.

- **Using an existing key pair** — import a public key with `aws ec2 import-key-pair` if you generated the pair locally.

```
# Create a new key pair via the AWS CLI
aws ec2 create-key-pair \
  --key-name my-ec2-key \
  --query 'KeyMaterial' \
  --output text > my-ec2-key.pem

# Import an existing public key
aws ec2 import-key-pair \
  --key-name my-imported-key \
  --public-key-material fileb://~/.ssh/id_ed25519.pub
```

Important: AWS does not store the private key. If you lose the `.pem` file, you cannot recover it. You must create a new key pair and update the instance.

2.2 Setting File Permissions

The private key file must have restrictive permissions, or SSH will refuse to use it. The only acceptable mode is `0600` (read and write by owner only).

```
# Set the correct permissions on the PEM file
chmod 600 my-ec2-key.pem

# Verify
ls -l my-ec2-key.pem
# Output: -rw----- 1 user user 1679 Jan  1 00:00 my-ec2-key.pem
```

2.3 Connecting with the PEM File

The SSH command uses the `-i` flag to specify the private key file. The default username depends on the AMI:

```
# Amazon Linux / RHEL
ssh -i my-ec2-key.pem ec2-user@<public-dns>

# Ubuntu
ssh -i my-ec2-key.pem ubuntu@<public-dns>

# Debian
ssh -i my-ec2-key.pem admin@<public-dns>

# CentOS
ssh -i my-ec2-key.pem centos@<public-dns>

# Fedora
ssh -i my-ec2-key.pem fedora@<public-dns>
```

No password is requested because the PEM file serves as the private key.

2.4 Copying the Public Key with `ssh-copy-id`

To enable passwordless access for a *different* key pair — for example, a key you generated locally for Ansible — use `ssh-copy-id`. This command appends your local public key to the remote `authorized_keys` file.

```
# Generate a local key pair for Ansible (if you have not already)
ssh-keygen -t ed25519 -f ~/.ssh/ansible_key -C "ansible-control"

# Copy the public key to the EC2 instance using the PEM file for authentication
ssh-copy-id -f "-o IdentityFile ~/.ssh/my-ec2-key.pem" \
  ubuntu@<instance-public-ip>

# Verify passwordless access with the new key
ssh -i ~/.ssh/ansible_key ubuntu@<instance-public-ip>
```

The `-f` flag forces the copy even if the key is already present. After this step, the `ansible_key` can be used for passwordless access, and the PEM file is no longer required for everyday automation.

3. EC2 Instance Setup via Password

Some workflows require password-based authentication, either as a bootstrap mechanism or because a policy mandates it. This section covers enabling password authentication on an EC2 instance and using it to install an SSH key for subsequent passwordless access.

Important: Password authentication is less secure than key-based authentication. Enable it only temporarily for the initial key deployment, then disable it.

3.1 Enabling Password Authentication

By default, Amazon Linux and Ubuntu AMIs disable password authentication. To enable it, edit the SSH daemon configuration on the instance.

```
# Log in to the instance using the PEM file
ssh -i my-ec2-key.pem ubuntu@<public-dns>

# Edit the SSH configuration
sudo vi /etc/ssh/sshd_config

# Find and change this line:
# PasswordAuthentication no
# to:
PasswordAuthentication yes

# Also ensure this line is present and not commented:
PubkeyAuthentication yes

# On Ubuntu, the cloud image may have an override file. Check and edit:
sudo vi /etc/ssh/sshd_config.d/60-cloudimg-settings.conf
# Change PasswordAuthentication no to yes

# Restart the SSH service
# On Amazon Linux:
sudo systemctl restart sshd

# On Ubuntu:
sudo systemctl restart ssh
```

3.2 Setting a Password on the Instance

Set a password for the default user. This password will be used once to copy the public key, then can be disabled.

```
# Set a password for the ubuntu (or ec2-user) account
sudo passwd ubuntu

# You will be prompted to enter and confirm the new password.
# Choose a strong, unique password.
```

3.3 Copying the Public Key with a Password

From the control node, use `ssh-copy-id` with password authentication. You will be prompted for the password you just set.

```
# Copy the public key using password authentication
ssh-copy-id ubuntu@<instance-public-ip>

# You will be prompted:
# ubuntu@<instance-public-ip>'s password: <enter the password>

# The command appends ~/.ssh/id_ed25519.pub (or id_rsa.pub) to the
# remote ~/.ssh/authorized_keys file.
```

3.4 Testing Passwordless Access

```
# Test that passwordless SSH now works
ssh ubuntu@<instance-public-ip>

# If no password is requested, passwordless authentication is working.

# You can now disable password authentication on the instance:
sudo vi /etc/ssh/sshd_config
# Change PasswordAuthentication yes back to no
sudo systemctl restart sshd

# Remove the password you set earlier:
sudo passwd -l ubuntu
```

With password authentication disabled, the only way to log in is with an authorised SSH key. This is the recommended final state for any production system.

4. Passwordless Authentication through Ansible

Ansible is agentless: it connects to managed nodes over SSH. For automation to work without manual intervention, Ansible must be able to authenticate without prompting for a password. This section covers the four mechanisms Ansible provides for passwordless SSH.

4.1 Ansible Inventory Configuration

The inventory file tells Ansible which hosts to manage and how to connect to them. When using key-based authentication, the private key path can be set at the group or host level.

```
# inventory/hosts.ini

[webservers]
web01.example.com ansible_host=192.0.2.10
web02.example.com ansible_host=192.0.2.11

[databases]
db01.example.com ansible_host=192.0.2.20

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/ansible_key
ansible_python_interpreter=/usr/bin/python3
```

With this configuration, Ansible will use the `ansible_key` private key for every host in the inventory. No password is ever requested.

4.2 Using `ansible_ssh_private_key_file`

The `ansible_ssh_private_key_file` variable can be set at different levels with different scopes.

```
# Per-host key
web01.example.com ansible_ssh_private_key_file=~/.ssh/web_key.pem

# Per-group key
[databases:vars]
ansible_ssh_private_key_file=~/.ssh/db_key.pem

# Global default (all hosts)
[all:vars]
ansible_ssh_private_key_file=~/.ssh/default_key.pem
```

4.3 Using `ssh-agent`

The SSH agent holds decrypted private keys in memory and supplies them to SSH clients on demand. Ansible can use the agent instead of specifying a key file explicitly. This is useful when keys are passphrase-protected.

```
# Start the SSH agent
eval $(ssh-agent -s)

# Add the private key (you will be prompted for the passphrase once)
ssh-add ~/.ssh/ansible_key

# Verify the key is loaded
ssh-add -l

# Run Ansible without specifying a key file
ansible all -i inventory/hosts.ini -m ping
```

With the agent running and the key loaded, Ansible connects without any password prompt. The private key remains encrypted on disk; only the agent holds the decrypted form.

4.4 Configuring `ansible.cfg` for Passwordless Access

A project-local `ansible.cfg` centralises connection settings so they do not need to be repeated on every command.

```
# ansible.cfg

[defaults]
inventory = ./inventory/hosts.ini
remote_user = ubuntu
private_key_file = ~/.ssh/ansible_key
host_key_checking = False
forks = 20

[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s

[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False
```

Note: `host_key_checking = False` is convenient in development but weakens security. In production, manage `known_hosts` explicitly.

4.5 Disabling Password Authentication on All Nodes

Once passwordless access works, disable password authentication on every managed node. Ansible can do this at scale with a simple playbook.

```
---
- name: Harden SSH on all managed nodes
  hosts: all
  become: true
  tasks:
    - name: Disable password authentication
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: "PasswordAuthentication no"
        validate: '/usr/sbin/sshd -t -f %s'
        notify: Restart sshd

    - name: Disable root login via SSH
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin'
        line: "PermitRootLogin no"
        validate: '/usr/sbin/sshd -t -f %s'
        notify: Restart sshd

  handlers:
    - name: Restart sshd
      ansible.builtin.service:
        name: sshd
        state: restarted
```

The validate parameter ensures the SSH configuration is syntactically correct before it is written. If the validation fails, the original file is left untouched.

4.6 Testing the Connection

```
# Ping all hosts to verify connectivity
ansible all -m ping

# Run a simple command
ansible all -m shell -a "hostname && uptime"

# Test a specific group
ansible webservers -m ping

# Expected output for each host:
# web01.example.com | SUCCESS => {
#   "changed": false,
#   "ping": "pong"
# }
```

5. Security Best Practices

5.1 Key Management

- **Use Ed25519 keys.** They are smaller, faster, and as strong as 3072-bit RSA. Generate with `ssh-keygen -t ed25519`.
- **Protect private keys with a passphrase.** Use `ssh-agent` to avoid retyping the passphrase.
- **Never share private keys.** Each user and each automation service should have its own key pair.
- **Rotate keys periodically.** Remove old public keys from `authorized_keys` when they are no longer needed.
- **Use a dedicated automation key.** Do not use a personal key for Ansible. Generate a separate key pair for automation.

5.2 SSH Configuration Hardening

- **Disable password authentication.** Set `PasswordAuthentication no` in `/etc/ssh/sshd_config`.
- **Disable root login.** Set `PermitRootLogin no`.
- **Use only SSH protocol 2.** Modern OpenSSH does this by default.
- **Restrict the SSH port** to trusted IP ranges in the security group or firewall.
- **Keep OpenSSH updated.** Apply security patches promptly.

5.3 Ansible-Specific Practices

- **Store private keys outside the repository.** Reference them with `ansible_ssh_private_key_file` or use `ssh-agent`.
- **Use ansible-vault for sensitive variables.** Do not store passwords in plain-text inventory files.
- **Prefer key-based authentication for all automation.** Password-based authentication should be a bootstrap mechanism only.
- **Use `ControlPersist`.** It keeps the SSH connection open between tasks, improving performance.
- **Enable pipelining.** It reduces the number of SSH round trips per task.

6. Troubleshooting

Symptom	Cause	Resolution
Permission denied (publickey)	Private key not readable or wrong permissions.	Run <code>chmod 600</code> on the key file.
Identity file not accessible	Path to the key is wrong.	Use an absolute path or <code>~/</code>
No identities found	<code>ssh-copy-id</code> cannot find a public key.	Generate a key pair first with <code>ssh-keygen</code> .
Connection timed out	Security group does not allow SSH.	Add an inbound rule for TCP port 22.

Symptom	Cause	Resolution
Host key verification failed	Server key changed or known_hosts stale.	Remove the old entry from known_hosts.
Ansible prompts for password	Key not specified or agent not running.	Set <code>ansible_ssh_private_key_file</code> or start <code>ssh-agent</code> .
sudo requires password	Passwordless sudo not configured.	Add <code>ansible</code> user to <code>/etc/sudoers.d/</code> .

Table 1 — Common passwordless authentication problems.

7. Quick Reference

Task	Command / Setting
Generate an SSH key pair	<code>ssh-keygen -t ed25519 -f ~/.ssh/key</code>
Set PEM permissions	<code>chmod 600 my-key.pem</code>
Connect with PEM file	<code>ssh -i key.pem ubuntu@host</code>
Copy key with PEM auth	<code>ssh-copy-id -f "-o IdentityFile key.pem" user@host</code>
Copy key with password auth	<code>ssh-copy-id user@host</code>
Enable password auth on EC2	<code>PasswordAuthentication yes</code> in <code>sshd_config</code>
Disable password auth on EC2	<code>PasswordAuthentication no</code> in <code>sshd_config</code>
Restart SSH (Amazon Linux)	<code>sudo systemctl restart sshd</code>
Restart SSH (Ubuntu)	<code>sudo systemctl restart ssh</code>
Start ssh-agent	<code>eval \$(ssh-agent -s)</code>
Add key to agent	<code>ssh-add ~/.ssh/key</code>
Ansible key in inventory	<code>ansible_ssh_private_key_file=~/.ssh/key</code>
Ansible key in ansible.cfg	<code>private_key_file = ~/.ssh/key</code>
Test Ansible connectivity	<code>ansible all -m ping</code>
Disable password auth via Ansible	<code>lineinfile</code> with <code>PasswordAuthentication no</code>

Table 2 — Passwordless authentication quick reference.

This document describes SSH passwordless authentication as implemented by OpenSSH and Ansible Core 2.14 and later. AWS EC2 default usernames and cloud image configurations may vary by region and AMI release; always consult the official AWS documentation for the AMI in use.